

Athenaeum

Library Management System — How We Built It

A step-by-step walkthrough: every step, why we did it,
and the concept behind it.

DBMS Capstone · MySQL · deployed on Vercel + Aiven

This document walks through the project in the order we built it — from the first idea to a live website. Each step states **What** we did, **Why**, and explains the **Concept** where a new idea appears. 47 steps across 10 phases.

PHASE 1 — Understanding the Problem & Planning

1 Choose and understand the project

What: We chose the **Library Management System** and wrote, in plain English, everything a library does: hold books, track copies, register members, lend/return books, charge late fines, and take reservations.

Why: A database is a mirror of a real process — you must understand the process first.

2 Find the entities (the "nouns")

What: We underlined the key nouns — Publisher, Category, Author, Book, Copy, Member, Staff, Borrowing, Fine, Reservation — each becomes a table.

Concept — Entity: a real-world thing we store data about. Each entity becomes a **table**; one row = one instance.

3 Decide the relationships

What: A publisher has many books; a book has many authors and vice-versa; a book has many copies; a member has many loans; a loan may have one fine.

Concept — Cardinality: how many rows relate on each side — **1:1**, **1:M**, **M:N**. We included all three.

4 Choose MySQL + InnoDB

What: MySQL as the database, InnoDB as the engine for every table.

Concept — Storage engine: InnoDB supports **foreign keys** and **transactions** (ACID); the older MyISAM does not.

5 Decide the 3-tier architecture

What: Browser UI → small server (Node/Express) → MySQL.

Why: A browser cannot reach MySQL directly, so a server sits in between.

Concept — 3-tier architecture: Presentation (browser) → Application (API) → Data (database). We later host these on **Vercel** (app) and **Aiven** (database).

PHASE 2 — Designing the Schema (on paper first)

6 Draw the ER diagram

What: An Entity–Relationship diagram of all 11 entities and the lines between them.

Concept — ER model: boxes (entities), their attributes, and lines (relationships). Crow's-foot notation marks the "many" side.

7 Choose surrogate primary keys

What: Each table gets an auto-increment id (`book_id` , `member_id` ...).

Concept — Primary & surrogate key: the PK uniquely identifies a row and can't be NULL; a surrogate key is an artificial id used because it never changes.

8 Separate Book (title) from Book Copy (physical item)

What: `book` holds title facts once; `book_copy` holds each physical copy with its own barcode and status.

Concept — Avoiding redundancy: store each fact in exactly one place — the core of good design.

9 Resolve M:N with a junction table

What: `book_author` holds `(book_id, author_id)` pairs.

Concept — Junction table: the standard way to implement many-to-many; its PK is the **composite** of two foreign keys.

10 Model the recursive category tree

What: `category.parent_id` points back to `category` (Physics → Science).

Concept — Recursive relationship: a table related to itself via a foreign key to its own PK.

11 Model the 1:1 Fine

What: `fine.borrow_id` is a foreign key that is also **UNIQUE**.

Concept — 1:1 relationship: a UNIQUE foreign key lets each loan have at most one fine.

PHASE 3 — Creating the Database (sql/01_schema.sql)

12 Create the database

```
CREATE DATABASE library_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

What: A fresh database with full-Unicode support.

Concept — DDL: Data Definition Language (CREATE, ALTER, DROP) defines structure.

13 Create publisher

```
publisher_id INT AUTO_INCREMENT PRIMARY KEY,  
name VARCHAR(120) NOT NULL UNIQUE, country VARCHAR(60) DEFAULT 'India'
```

Concept — Constraints: PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, AUTO_INCREMENT.

14 Create category with a self-FK

```
FOREIGN KEY(parent_id) REFERENCES category(category_id) ON DELETE SET NULL
```

Concept — Referential actions: CASCADE / SET NULL / RESTRICT decide what happens to child rows.

15 Create author with a CHECK

```
CHECK (birth_year IS NULL OR birth_year BETWEEN 1000 AND 2025)
```

Concept — Domain integrity / CHECK: keeps a value within a valid range.

16 Create `book` with two foreign keys

```
FOREIGN KEY(publisher_id) REFERENCES publisher(publisher_id) ON DELETE RESTRICT
```

Concept — Referential integrity: a foreign key must match an existing primary key (or be NULL).

17 Create the `book_author` junction

```
PRIMARY KEY (book_id, author_id)
```

Concept — Composite key: a primary key of two columns; the pair must be unique.

18 Create `book_copy` with an ENUM status

```
status ENUM('Available', 'Borrowed', 'Reserved', 'Lost', 'Damaged')
```

Concept — ENUM: restricts a column to a fixed list of values.

19 Create `member` with a soft-delete flag

```
email VARCHAR(120) NOT NULL UNIQUE, is_active BOOLEAN DEFAULT TRUE
```

Concept — Soft delete: deactivate instead of removing, to keep history valid.

20 Create `staff`

What: The librarians who issue books, so each loan records who processed it.

21 Create `borrowing` — the central transaction table

```
borrow_date DATE, due_date DATE, return_date DATE NULL, -- NULL = still out  
CHECK(due_date >= borrow_date)
```

Concept — NULL: means "not yet". A NULL return date marks an active loan.

22 Create `fine` (1:1)

```
borrow_id INT NOT NULL UNIQUE
```

23 Create `reservation`

What: Lets a member reserve a title (Pending/Fulfilled/Cancelled).

24 Add indexes

```
CREATE INDEX idx_book_title ON book(title);
```

Concept — Index: a sorted lookup that speeds searches; costs space and slightly slows writes.

25 Demonstrate ALTER TABLE

```
ALTER TABLE member ADD COLUMN last_updated TIMESTAMP ...
```

Concept — ALTER: a DDL command that changes an existing table.

PHASE 4 — Checking Normalization

26 First Normal Form (1NF)

What: Every cell atomic — multi-valued authors/copies moved to their own tables.

Concept — 1NF: atomic values, no repeating groups.

27 Second Normal Form (2NF)

What: In `book_author`, the only non-key column depends on the whole composite key.

Concept — 2NF / partial dependency: no non-key column may depend on part of the key.

28 Third Normal Form (3NF)

What: No non-key column depends on another non-key column (publisher city lives in `publisher`, not `book`).

Concept — 3NF / transitive dependency: every non-key attribute depends only on the key — no redundancy, no anomalies.

PHASE 5 — Inserting Sample Data (sql/02_data.sql)

29 Insert reference data first

What: Publishers, categories, authors before books.

Concept — Insertion order: parent (referenced) tables load before child tables, or the FK check fails.

30 Insert books + author links

What: 25 books, then the `book_author` pairs (some with multiple authors).

31 Insert copies, members, staff

What: 40 copies (some Lost/Damaged), 20 members, 5 staff.

32 Insert borrowings — a deliberate mix

What: 30 loans: returned, returned-late, active, and overdue.

Why: So the overdue/fine views return real results in the demo.

33 Insert fines and reservations

What: Completes the dataset to ~200 records (over the 100 minimum).

Concept — DML: Data Manipulation Language (INSERT/UPDATE/DELETE/SELECT).

PHASE 6 — Views (sql/03_views.sql)

34 Create `vw_book_catalogue`

```
GROUP_CONCAT(... SEPARATOR ', ') AS authors
```

What: Joins book+category+publisher+authors into one row.

Concept — View: a stored SELECT used like a virtual table; hides complex joins, enables reuse.

35 Availability, active-loans, fines views

What: Power the dashboard and tables directly.

Concept — Derived attribute: "days overdue" is computed on the fly with `DATEDIFF(CURDATE(), due_date)`, never stored.

PHASE 7 — Advanced Queries (sql/04_queries.sql)

36 JOIN queries

What: Loans with member+title, members who never borrowed (LEFT JOIN), sub-categories beside parents (self join).

Concept — JOIN types: INNER, LEFT, SELF — we use all three.

37 Aggregates + GROUP BY / HAVING

```
GROUP BY b.book_id HAVING COUNT(br.borrow_id) >= 2
```

Concept — WHERE vs HAVING: WHERE filters rows before grouping; HAVING filters groups after.

38 Nested / correlated subqueries

```
WHERE price > (SELECT AVG(price) FROM book)      -- scalar
WHERE EXISTS (SELECT 1 FROM borrowing ... )      -- correlated
```

Concept — Subquery types: scalar, correlated (EXISTS), and IN / NOT IN.

PHASE 8 — Logic in the Database (sql/05_procedures.sql)

39 Triggers

```
IF v_status <> 'Available' THEN SIGNAL ... -- block bad loan
```

What: Block issuing an unavailable copy; auto-mark a copy Borrowed on loan; auto-create a fine + free the copy on a late return.

Concept — Trigger: code that runs automatically on INSERT/UPDATE/DELETE. **OLD / NEW** hold the row before/after; **SIGNAL** raises an error.

40 Stored procedures + transactions

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
START TRANSACTION; ... COMMIT;
```

What: `sp_issue_book`, `sp_return_book`, `sp_pay_member_fines`, plus a function `fn_member_balance`.

Concept — Transaction & ACID: all-or-nothing (Atomicity), valid (Consistency), isolated (Isolation), permanent once committed (Durability).

PHASE 9 — Building the Application (Athenaeum)

41 Build the backend API (Express)

```
app.get('/api/books', (req,res) => pool.query('SELECT * FROM vw_book_catalogue ...') ...);  
app.post('/api/loans', ... 'CALL sp_issue_book(?,?,?,?,@bid)' ...);
```

What: REST endpoints that read the views and call the stored procedures, returning JSON.

Concept — API / REST: URLs the frontend calls (e.g. `/api/books`) to get/change data.

42 Build the front-end (vanilla HTML/CSS/JS)

What: A single page named **Athenaeum** — Home, Catalogue, On Loan, Issue, Members, Fines.

Why: The brief recommends HTML/CSS/JS and a simple UI keeps the focus on the database.

43 Design a warm, handmade look

What: An editorial theme — a script logo, Playfair serif headings, a hero banner with an overlaid panel, a cream paper palette, and clean white tables.

Why: A considered, human design earns the "user-friendly design" credit and reads as real, not generic.

Concept — Separation of concerns: structure (HTML), style (CSS) and behaviour (JS) live in separate files, so the look can change without touching the logic.

44 Connect front-end to back-end with fetch()

```
const rows = await fetch('/api/books').then(r => r.json());
```

What: JavaScript requests data and renders tables; Issue/Return POST back. Trigger errors (e.g. "Copy not available") flow back and are shown to the user.

Concept — fetch / AJAX: JavaScript talking to the server in the background to update the page.

PHASE 10 — Hosting it Online

45 Run locally with Docker (development)

```
docker compose up -d    # MySQL + auto-loads sql/
npm start                # http://localhost:3000
```

Concept — Docker / container: a portable package that runs MySQL the same way on any machine.

46 Host the database on Aiven (cloud MySQL)

```
# import the combined script into the cloud DB
docker run --rm -i -v "$PWD/sql:/s" mysql:8.0 \
  sh -c "mysql -h HOST -P PORT -u avnadmin -pPASS --ssl-mode=REQUIRED < /s/init.sql"
```

What: Created a free MySQL service on Aiven and imported `sql/init.sql` (schema + data + views + procedures).

Why: Vercel cannot host a database, so the data lives on a real cloud MySQL that supports foreign keys.

Concept — Managed database / TLS: a cloud-run database; connections are encrypted, so the app connects with SSL enabled.

47 Deploy the app on Vercel

```
# Vercel reads these environment variables (set in the dashboard):  
MYSQL_URL = mysql://avnadmin:****@HOST:PORT/library_db  
DB_SSL    = true
```

What: Imported the GitHub repo into Vercel; `vercel.json` serves `public/` and routes `/api/*` to the serverless function in `api/index.js`. The database URL is supplied as an environment variable, then we redeploy.

Why: This gives a public URL where the live front-end talks to the live database — the full 3-tier system, online.

Concept — Serverless & environment variables: the API runs on demand (no always-on server); secrets like the DB URL are injected as environment variables, never hard-coded.

— End of Walkthrough · 47 steps across 10 phases —